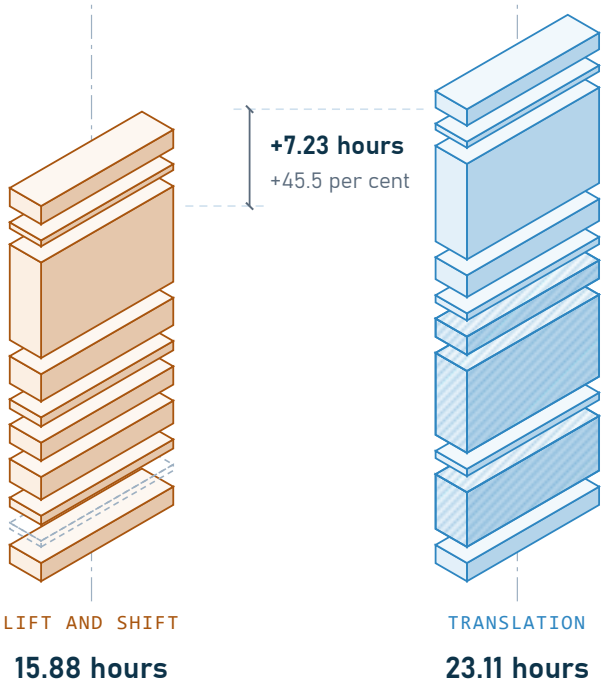
NORTHMERE COMPONENTS GROUP
RESEARCH REPORT, NOVEMBER 2026

Counting before choosing

Object-based estimation for a data-warehouse migration

A case study in bottom-up effort estimation

ONE OBJECT, TEN TASKS, TWO ROUTES



THE WHOLE ESTATE

286
storage objects

521.6
man-days, lift and shift

675.6
man-days, translation

592.5
man-days, the hybrid

Tariq Hussain
Data Platform Operations
November 2026

FOREWORD

An estimate that can be argued with

A note on the data

Every figure in this paper describes a fictitious organisation. The method is real; the numbers are not.

I have been handed a number for a migration more than once, and the number was never the problem. The problem was that nobody in the room could say where it came from. A total arrives with a confidence that its parts would never support, and the discussion that follows is about the person who produced it rather than about the work it describes.

This paper sets out the opposite habit. It counts the objects in a warehouse, scores each one against ten tasks, converts the scores to hours through a matrix that has been calibrated against what the original build actually cost, and adds the result up. Nothing in that chain is clever. What it buys is the ability to be challenged line by line: a reader who disagrees can point at a score, at a task or at the matrix, and the disagreement becomes a question with an answer.

The case is a decision between rehosting a warehouse as it stands and rebuilding it in the pattern of the platform it is moving to. The method puts the second route 29.5 per cent above the first and, more usefully, says where that difference sits: three of the ten tasks carry all of it. It also shows that the judgement behind the first draft of the matrix fell 22.8 per cent short of what the original build cost, which is the error that calibration exists to remove.

I would rather be wrong in a way that can be found than right in a way that cannot be checked. That preference is the whole argument of what follows, and I have written it down so that I hold myself to it the next time a total arrives without its parts.

Tariq Hussain

CONTENTS

What follows

01 **Introduction** 04

1.1 Purpose 1.2 Scope 1.3 The decision Northmere faces 1.4 Structure of the report

02 **Estimating bottom-up** 06

2.1 Four families of estimate 2.2 Top-down and bottom-up
2.3 Why the storage object is the unit 2.4 The planning fallacy and the outside view

03 **The case** 09

3.1 Northmere and the convergence mandate 3.2 ATLAS, built in two steps
3.3 HORIZON, built in one 3.4 Two routes 3.5 The inventory 3.6 The decision to be made

04 **The method** 12

4.1 The chain 4.2 Ten tasks 4.3 The rubric 4.4 Drivers and the scoring guide
4.5 The lookup matrices 4.6 Aggregation and the architectural component
4.7 One object through both routes

05 **Calibration** 19

5.1 The reference class 5.2 The stratified sample 5.3 The ratio estimator
5.4 Bias and error 5.5 The bootstrap interval 5.6 The diagnostic by type
5.7 The calibrated and target matrices

06 **Results** 22

6.1 Two totals 6.2 Where the difference sits: by task 6.3 By object type
6.4 By subject area 6.5 A Pareto view of the objects 6.6 Duration and staffing

07 **The hybrid route** 26

7.1 The premium object by object 7.2 The unit of decision 7.3 The decision rule
7.4 The hybrid estimate 7.5 What each route buys and costs

08 **Sensitivity and limitations** 29

8.1 One metric 8.2 Five cases 8.3 Why the scores dominate
8.4 What the method cannot see 8.5 Two scorers

09 **Governing the estimate** 32

9.1 The score record 9.2 Re-scoring as discovery proceeds
9.3 Tracking actuals against the estimate 9.4 The estimate in a steering decision

10 **Conclusion** 34

A **Appendix** 35

A.1 Task definitions A.2 The rubric and the build route A.3 The lookup matrices
A.4 An inventory excerpt A.5 The formulas A.6 Glossary
A.7 References and further reading A.8 A note on the fictitious data

CHAPTER ONE

01 Introduction

Two routes for moving a warehouse, one method for pricing them, and the reason the method counts objects before it estimates anything.

1.1 Purpose

This paper describes a method for estimating the effort of migrating a data warehouse, and applies it to a decision between two routes. The method is bottom-up. It estimates every storage object in the warehouse separately, against the same ten tasks, and adds the results. Its purpose is not only to produce a total but to produce one that a reader can take apart: every man-day in the result can be traced to a score on a named object, and every score to a rubric that a second person can apply.

The paper is the practical sequel to an earlier one on top-down and bottom-up estimation (Hussain, 2022), which compared the two approaches in general terms. Here the bottom-up approach is followed through one decision from start to finish: the inventory, the scoring, the calibration against what the original build cost, the comparison of the routes, a third route derived from that comparison, and an account of how far the answer depends on the inputs that are least certain.

1.2 Scope

The paper covers four pieces of work, which together make up the estimate.

- **The estimate per object.** Ten tasks, a rubric of six levels, the drivers that move a score, and two lookup matrices that convert scores into hours.
- **Calibration.** The adjustment of the matrices against the recorded effort of the original build, with its error before and after.
- **The routes.** The comparison of lift and shift with translation, and the hybrid route that the comparison suggests.
- **Duration, sensitivity and governance.** Calendar time for a team of a given size, the inputs to which the answer is most sensitive, and how the estimate is kept honest once the work begins.

The paper does not price licences or infrastructure, which are bought rather than built, and it keeps out of the object model the work that belongs to no single object, such as environments, security and programme management. Section 8.4 explains why that work is estimated separately and from the top down.

Fictitious data

Northmere, ATLAS, HORIZON and every number in this paper are invented. Appendix A.8 sets out how the data were generated.

The organisation, its systems and its figures are fictitious. The method is one the author has used in practice, generalised here so that it can be described in full and reproduced from the model that accompanies the paper.

1.3 The decision Northmere faces

Northmere Components Group, a European distributor of industrial components, has asked each of its regional organisations to move its reporting onto one group data platform. Northmere West runs its own warehouse, ATLAS, built as a two-step Data Vault 2.0 design on Microsoft SQL Server and SSIS. The group platform, HORIZON, loads and transforms data in a single step on Azure Databricks, with dbt for the models and Azure Data Factory for ingestion.

Two routes

Route A rehosts ATLAS unchanged. Route B rebuilds every object in HORIZON's pattern. Chapter 3 defines both.

Two routes were on the table. The first, lift and shift, moves ATLAS onto cloud infrastructure as it stands, with SQL Server, SSIS and the two-step design intact. The second, translation, rebuilds every object in the single-step pattern that HORIZON expects. The first is cheaper and leaves Northmere West outside the group's pattern; the second converges at once and costs more. The question for the steering group was how much more, and where the extra effort would go.

A top-down estimate would not settle that question. One figure for each route, taken from another migration or from a rule of thumb, would give two totals and no account of the difference between them. The two routes share most of their work: the same sources must be understood, the same data reconciled and the same reports cut over. The difference sits in a few tasks on some objects, and only an estimate built object by object and task by task can show where.

1.4 Structure of the report

The report is arranged in ten chapters and an appendix, in the order in which the estimate is built.

- **Chapter 2** places object-based estimation among the families of estimating method and introduces the bias that calibration corrects.
- **Chapter 3** describes Northmere, the two architectures and the inventory.
- **Chapter 4** sets out the method and takes one object through both routes.
- **Chapter 5** calibrates the matrix against the original build.
- **Chapters 6 and 7** give the results and derive the hybrid route.
- **Chapter 8** tests the answer against its least certain inputs and names what the method cannot see.
- **Chapters 9 and 10** describe how the estimate is governed and draw the conclusions.
- **The appendix** holds the task definitions, the rubric, the matrices, an excerpt of the inventory, the formulas, a glossary and the references.

CHAPTER TWO

02 Estimating bottom-up

Four families of estimate, the place of object-based estimation among them, and the bias that calibration exists to remove.

2.1 Four families of estimate

Methods for estimating project effort fall into four broad families, which the PMBOK Guide describes among its tools for estimating durations and costs (Project Management Institute, 2017). Each family answers the question of how much effort from a different kind of evidence, and each suits a different stage of knowledge. Table 2.1 sets them side by side.

Family	Evidence it uses	Strength	Weakness
Expert judgement	The experience of people who have done similar work	Fast, and uses knowledge that is hard to write down	Cannot be inspected, and inherits the optimism of whoever gives it
Analogous	The actual effort of a comparable past project	Quick, and anchored in an outcome rather than a hope	Only as good as the comparison between the two projects
Parametric	A statistical relationship between effort and a measured size	Repeatable, and calibrated on many projects	The size measure may not describe the work in hand
Bottom-up	An estimate for every element of the work, summed	Detailed, and open to challenge item by item	Slow to build, and blind to work that is not listed

TABLE 2.1 The four families of estimating method and the evidence each relies on.

Parametric methods deserve a closer look, because the method in this paper borrows from them. COCOMO II expresses effort as a function of software size, adjusted by scale factors and effort multipliers that were fitted to a large database of completed projects (Boehm et al., 2000). Function point analysis measures size by counting what a system does for its users, independently of the technology it is written in (Albrecht, 1979; International Function Point Users Group, 2010). Both rest on the same idea: count something that can be counted without opinion, and let a calibrated relationship turn the count into effort.

McConnell (2006) puts the same idea as advice to the estimator: count if at all possible, compute from the counts where counting is not enough, and rely on judgement alone only where nothing can be counted. The four families are not rivals in that view. A mature estimate uses several of them, each for the part of the work where its evidence is strongest.

2.2 Top-down and bottom-up

The earlier paper set the families on a spectrum from top-down to bottom-up (Hussain, 2022). A top-down estimate starts from the whole, a total for the programme divided among its parts by proportion. A bottom-up estimate starts from the parts and adds them. Top-down estimates are quick and are right about the work that belongs to the whole, such as management and integration; bottom-up estimates are slower and are right about the work that belongs to the parts. The earlier paper concluded that most teams sit somewhere between the two, and that the useful question is which parts of an estimate belong at which end.

For a warehouse migration the answer is unusually clear. Most of the work belongs to objects: each table, vault structure and mart has to be understood, rebuilt or moved, loaded, tested, reconciled and deployed. That work belongs at the bottom-up end. The work that does not belong to any object, such as standing up environments or running the programme, belongs at the top-down end, and this paper estimates it there.

2.3 Why the storage object is the unit

Storage object

Anything the warehouse persists and names in its model: a staging table, a hub, a link, a satellite, a reference table, a mart or a view.

A bottom-up estimate needs a unit of work that can be counted without judgement and that carries effort roughly in proportion to its size. A report is a poor unit, because two reports over the same model can cost very different amounts. A pipeline is a poor unit, because one pipeline can load a single table or fifty. The storage object is a better unit than either, for four reasons, each of which the method relies on.

- **It can be counted by query.** Every object is named in the warehouse's catalogue, so the inventory is a fact rather than an opinion.
- **It is the unit of delivery.** Objects are what a team designs, builds, tests, reconciles and deploys, so the ten tasks attach to it naturally.
- **It is the unit of record.** Actual effort can be booked against an object, which is what makes calibration and tracking possible.
- **Its size is visible.** Attributes, sources, rules, history and dependants can be read from the model and the code, so the drivers of effort are observable.

Object-based estimation is therefore a bottom-up method with a parametric core. It counts objects as function point analysis counts functions, scores each object against a rubric, and converts the scores to hours through a lookup matrix that plays the part of a parametric model's calibrated relationship.

2.4 The planning fallacy and the outside view

Kahneman and Tversky, who introduced the term in 1979, gave the name planning fallacy to the tendency of forecasters to underestimate how long a task will take, even when they know that similar tasks have overrun (Kahneman and Tversky, 1982). Buehler, Griffin and Ross (1994) showed the effect experimentally: people asked to predict their own completion times were too optimistic, and remained so when reminded of their past performance.

The outside view

Start from the outcomes of a class of similar cases, then adjust for what is particular to this one.

The remedy Kahneman and Tversky proposed is the outside view. Instead of building a forecast from the particulars of the case in hand, the forecaster starts from the distribution of outcomes in a class of comparable cases and adjusts from there. Flyvbjerg (2006; 2008) turned that remedy into reference class forecasting and applied it to large infrastructure programmes, whose cost overruns had followed the same pattern for decades.

A bottom-up estimate is an inside view by construction, because it is built from the particulars of the case. It is exposed to exactly the bias that the outside view corrects. The method in this paper takes the outside view in one place, through calibration. The matrix of hours per task and level is first written from judgement, and is then scaled by the ratio of actual to predicted effort on a sample of objects whose real cost is known. In the case, that sample comes from the original build of ATLAS, which is the closest reference class available. Chapter 5 shows that the judgement behind the matrix fell 22.8 per cent short of it.

CHAPTER THREE

03 The case

A distributor, a warehouse built in two steps, a group platform built in one, and 286 objects to move.

3.1 Northmere and the convergence mandate

Northmere Components Group distributes industrial components across Europe through regional organisations, each of which grew its own reporting. Group IT has mandated that every region converge on one data platform, so that group figures are produced once, in one pattern, from one set of definitions. Northmere West is the first region whose warehouse must move, and its choice of route will be read by the regions that follow.

3.2 ATLAS, built in two steps

ATLAS follows the Data Vault 2.0 pattern described by Linstedt and Olschimke (2015). Data arrive in staging tables as they were received. A first step loads them into the raw vault, which separates business keys into hubs, relationships into links and descriptive attributes, with their history, into satellites, applying only hard rules such as type conversion. A second step applies the soft rules, the business logic, in a business vault, from which marts and views are built for reporting. Every step is an SSIS package running against SQL Server, and the upper half of Figure 3.1 draws the sequence.

3.3 HORIZON, built in one

HORIZON lands data through Azure Data Factory and transforms it in one step inside Azure Databricks. Each transformation is a dbt model, a SELECT statement that dbt materialises as a table or a view; dbt derives the order in which models run from the references between them, runs the tests declared beside each model, and generates documentation and lineage from the code. Business rules are applied once, in the model that needs them, rather than in a separate layer.

3.4 Two routes

The two routes differ in what they carry across, and the difference is what the estimate has to price.

- **Route A, lift and shift.** ATLAS is rehosted on cloud infrastructure with SQL Server, SSIS and the two-step design intact. Nothing is redrawn; every object is moved, reconnected, reconciled and cut over as it stands.
- **Route B, translation.** Every object is rebuilt in HORIZON's single-step pattern. The vault structures are redesigned as dbt models, their load logic is

rewritten, and the business rules are re-expressed where the target applies them.

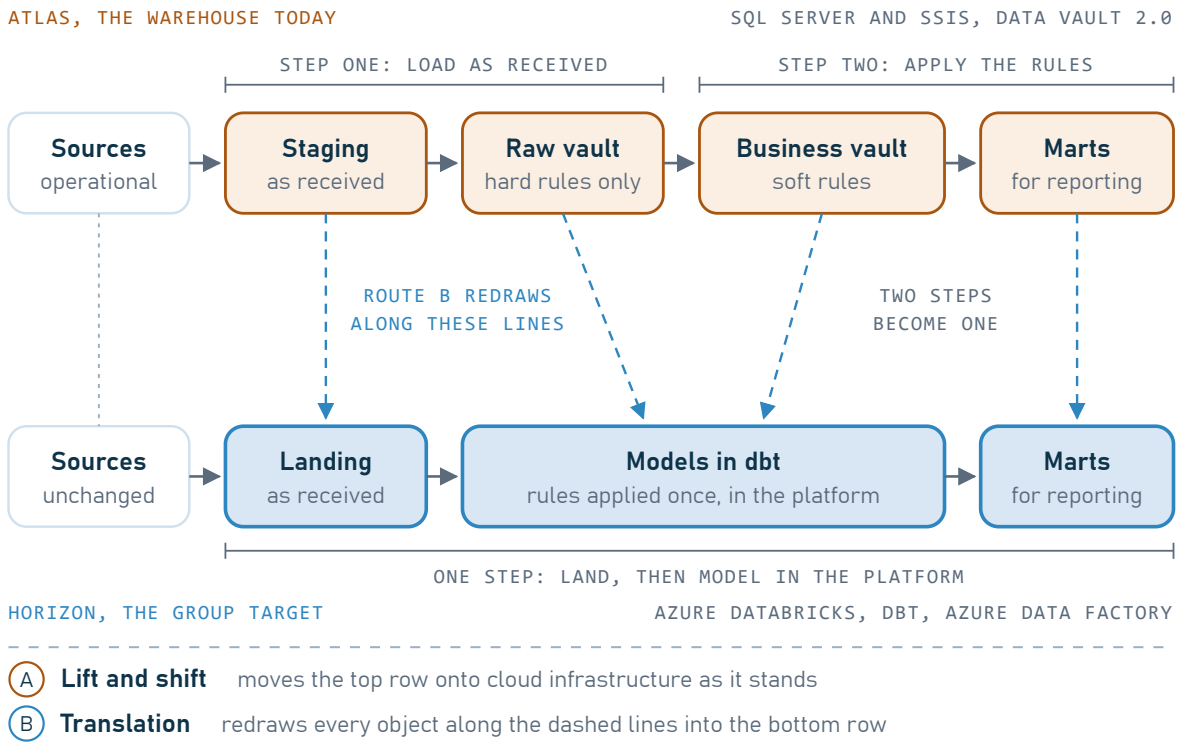


FIGURE 3.1 ATLAS, above, and the group target, HORIZON, below. The dashed lines are what the translation route redraws: the raw vault and the business vault collapse into one set of models.

Because route A keeps the design and the tools, the premium of route B over route A prices the change of pattern alone. A comparison with a route that also changed the database or the scheduler would mix two decisions into one figure, and the steering group could not then tell which of them it was paying for.

3.5 The inventory

ATLAS holds 286 storage objects in eight subject areas: 68 staging tables, 33 hubs, 30 links, 68 satellites, 24 reference tables and 63 marts or views. Table 3.1 shows how they divide among the areas. The mix matters, because a satellite carries history that the translation route must redesign, while a staging table is moved much as it is on either route.

Customer, Finance and Product are the most satellite-heavy areas, with 19, 15 and 14 satellites respectively. Sales, Logistics and Inventory are dominated by staging tables and marts, and reference data consists mostly of reference tables (17 of its 32 objects). Chapter 7 shows that this difference in mix is what separates the areas that are cheap to translate from those that are not.

For every object the inventory records seven drivers, read from the model and the code: the number of attributes, the number of sources and joins, the number of business rules, the kind of history kept, the data volume, the number of downstream dependants, and whether a data-quality issue is known. Chapter 4 explains how each driver moves the score.

Subject area	Staging	Hub	Link	Satellite	Reference	Mart or view	Objects
Sales	14	3	0	5	1	21	**44**
Customer	7	3	5	19	1	3	**38**
Product	5	5	10	14	1	1	**36**
Inventory	7	4	3	5	0	15	**34**
Purchasing	7	9	3	4	0	9	**32**
Logistics	18	3	0	2	1	6	**30**
Finance	3	6	9	15	3	4	**40**
Reference data	7	0	0	4	17	4	**32**
All areas	**68**	**33**	**30**	**68**	**24**	**63**	**286**

TABLE 3.1 Storage objects by subject area and type.

3.6 The decision to be made

The steering group needed four things from the estimate, and the rest of the paper supplies them in order: a total for each route; an account of where the difference between the totals sits; a duration for a plausible team; and a statement of how far the answer could move if the least certain inputs were wrong. A fifth followed from the first two, because a premium that is uneven across the estate invites a route that is neither of the original two.

CHAPTER FOUR

04 The method

Ten tasks, a six-level scale, the drivers that move a score, and a lookup matrix that turns a score into hours. The chapter ends by taking one object through both routes with the arithmetic shown.

4.1 The chain

The method is a chain of lookups and sums, shown in Figure 4.1. Each object is decomposed into the same ten tasks; each task is scored from 0 to 5 on each route; a lookup matrix converts each score into hours; the hours add up to the object, the objects to the route, the hours to man-days, and the man-days to months for a team of a given size. No step in the chain is a judgement except the score, which is why the score is where review is spent.

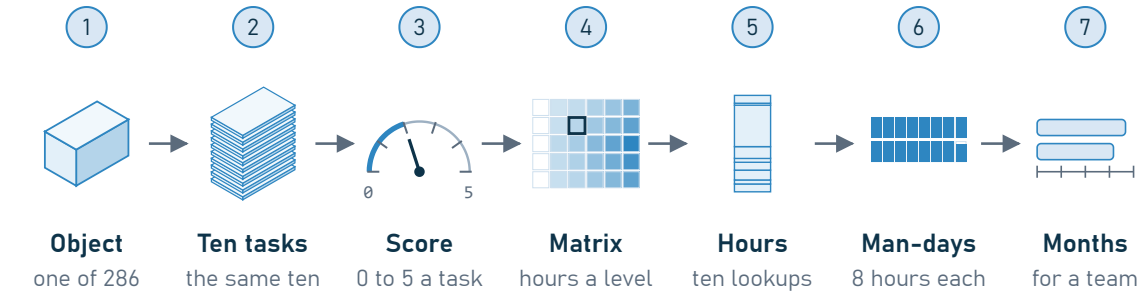


FIGURE 4.1 The chain from one object to a duration, worked underneath for the object followed in section 4.7, OBJ-088.

4.2 Ten tasks

Translation tasks

Tasks 2, 4 and 5 carry the change from two steps to one. On route A they are scored against the existing design; on route B against HORIZON's.

Every object passes through the same ten tasks, whichever route the programme takes. The tasks are deliberately ordinary, because a task that a delivery lead does not recognise cannot be scored consistently by two people. Table 4.1 lists them; Appendix A.1 defines each in full.

4.3 The rubric

Each task is scored on a scale of six levels, defined in Table 4.2. A score describes the work the task requires for that object on that route, not the object's importance or its size in rows. Level 0 is reserved for a task that needs no work at all, because it is reused or fully automated.

No	Task	What it covers
01	Source analysis and lineage	Understanding where the object's data come from and how they flow into it
02	**Target data model design**	Designing the object's structure, keys and history in the target pattern
03	Object build	Creating the physical structures on the platform
04	**Load logic development**	Writing the mappings or models that load the object
05	**Business rules and transformations**	Implementing the business logic the object carries
06	Orchestration and scheduling	Placing the load in the job chain with its dependencies
07	Unit testing	Testing the object's load and logic in isolation
08	Data reconciliation against the source	Proving that the object matches its source: counts, sums and history
09	Documentation and lineage registration	Recording the object's meaning and lineage in the catalogue
10	Deployment, cut-over and hypercare	Moving the object to production, switching over and supporting it

TABLE 4.1 The ten tasks. The three in bold carry the architectural translation.

Level	Meaning	Typical evidence
0	**No work**	The task is fully reused or automated, so nothing is done for this object
1	**Trivial**	A template applies with no decisions to make
2	**Simple**	Minor adjustments and few attributes
3	**Moderate**	Several sources or attributes, and standard historisation
4	**Complex**	Many sources, custom business rules, non-standard history or known data-quality issues
5	**Very complex**	Heavy logic, dependencies across objects, performance tuning or unclear requirements

TABLE 4.2 The rubric.

4.4 Drivers and the scoring guide

Scores are not free judgements. Each is anchored to the seven drivers recorded in the inventory, which are first turned into signals on a common scale (Table 4.3). A scoring guide then gives, for every task and route, a constant and a weight for each signal that affects it (Table 4.4). In practice a scorer reads the guide as a starting point and records a reason wherever the score departs from it; in the fictitious case the guide generates the scores, with a small random term standing in for the scorer's judgement.

Signal	Driver	Definition
a	Attributes	The number of attributes divided by 20, at most 3
s	Sources	The number of sources less one, divided by 2, at most 3
r	Rules	The number of business rules divided by 4, at most 3
h	History	None 0, overwrite (SCD1) 0.5, full history (SCD2) 1.2, multi-active 2
v	Volume	Low 0, medium 0.7, high 1.5
d	Dependants	The number of downstream dependants divided by 4, at most 3
q	Data quality	1 when a data-quality issue is known, otherwise 0

TABLE 4.3 The drivers, turned into signals.

Task	Lift and shift	Translation	Translation, vault type
01 Source analysis	0.60 + 1.08 d + 0.90 q	0.80 + 1.08 d + 0.90 q	0.80 + 1.08 d + 0.90 q
02 Model design	0.10	0.30 + 1.26 h	1.20 + 1.26 h
03 Object build	0.80 + 0.54 a	0.90 + 0.54 a	0.90 + 0.54 a
04 Load logic	0.90 + 0.90 s + 0.90 v	0.90 + 1.08 s + 1.08 h	1.80 + 1.08 s + 1.08 h
05 Business rules	0.10 + 0.72 r + 0.90 q	0.30 + 1.62 r + 0.90 q	0.60 + 1.62 r + 0.90 q
06 Orchestration	1.00 + 1.08 d	1.10 + 1.08 d	1.10 + 1.08 d
07 Unit testing	1.00 + 0.90 r + 0.54 h	1.10 + 0.90 r + 0.54 h	1.10 + 0.90 r + 0.54 h
08 Reconciliation	1.40 + 1.08 h + 1.80 v + 0.90 q	1.50 + 1.08 h + 1.80 v + 0.90 q	1.50 + 1.08 h + 1.80 v + 0.90 q
09 Documentation	0.50 + 0.36 a	0.60 + 0.36 a	0.60 + 0.36 a
10 Deployment	1.00 + 0.54 v + 0.72 d	1.10 + 0.54 v + 0.72 d	1.10 + 0.54 v + 0.72 d

TABLE 4.4 The scoring guide: the score for each task is the constant plus each weight times its signal, plus the scorer's judgement, rounded and held between 0 and 5. A vault type is a hub, a link or a satellite.

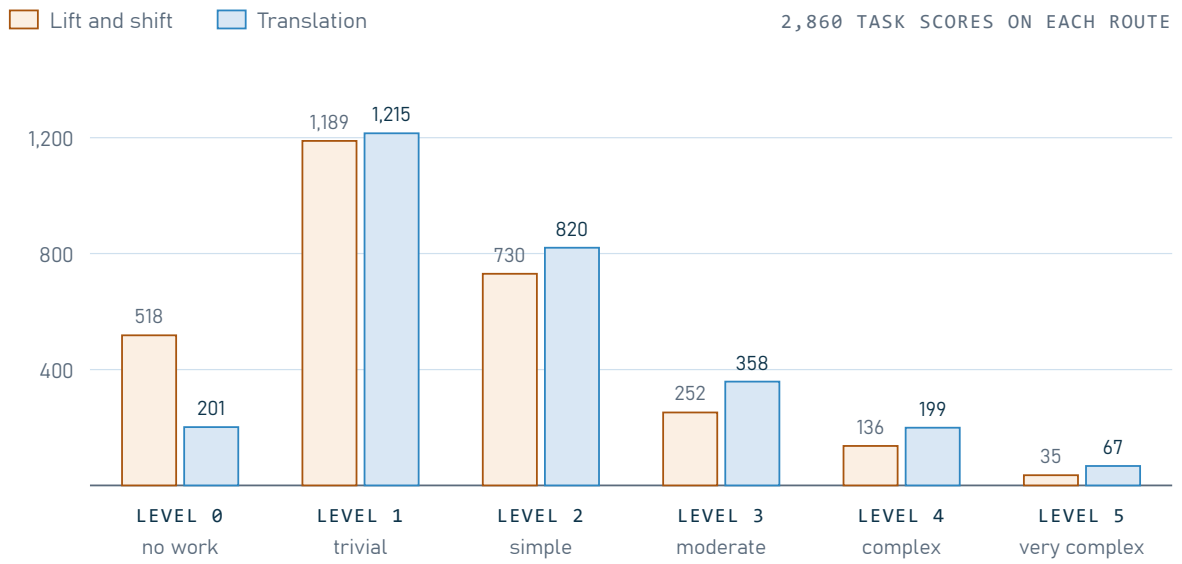
The guide encodes how each driver moves the score, and its pattern is the mechanism the rest of the paper relies on. On the lift-and-shift route, design and business rules are reused, so their constants are close to zero and no driver raises them much. Reconciliation carries the heaviest weights on that route, because a rehosted warehouse must prove that it produces the same numbers, and volume, history and known data-quality issues all make that proof harder.

On the translation route, history and the vault flag raise the design and load-logic scores, because a vault structure with history must be redesigned as a model that produces the same result in one step. Business rules weigh more than twice as heavily as they do on lift and shift, because every rule must be re-expressed and agreed again where the target applies it. The other tasks differ little between the routes, which is why the difference between them concentrates in three tasks.

Judgement

A shared term per object and task makes a hard object hard on every route; a small term per route adds the scorer's variation.

Figure 4.2 shows the result across the inventory. Translation has far fewer zero scores than lift and shift, because a task that is redrawn cannot be reused, and more scores at every level from 2 upwards.



Translation moves scores out of level 0 and up into levels 2 to 5, because a redrawn task cannot be reused.

FIGURE 4.2 The complexity distribution: all 2,860 task scores on each route, by level.

4.5 The lookup matrices

A lookup matrix converts a score into hours. Its ten rows are the tasks and its six columns the levels. Each cell is the task's base hours at level 3, scaled by a level curve that grows faster than the level, so that a level 5 task costs 12 times a level 1 task. Calibration then multiplies the whole matrix by one factor, k, derived in Chapter 5.

The translation route reads a second matrix, derived from the first through a relative-effort factor for each task. A factor below one marks a task that HORIZON makes cheaper at the same level of difficulty. Table 4.5 gives the base hours and the factors with the reason for each, and Figure 4.3 shows the calibrated legacy matrix with its level curve.

Task	Base hours	Factor	Reason for the factor
01 Source analysis	2	1.0	The same legacy sources must be understood on either platform.
02 Model design	3	1.0	Design effort per level is the same; the extra design of translation shows in its higher scores, not in the matrix.
03 Object build	1	0.8	dbt materialisations generate the structures, so less hand-written DDL is needed.
04 Load logic	5	0.9	A dbt model in SQL is slightly quicker to write than an SSIS data flow of equal complexity.
05 Business rules	4	1.0	Each rule must be re-expressed and agreed with the business on either platform.
06 Orchestration	1	0.8	dbt derives the run order from the references between models.
07 Unit testing	3	0.8	dbt tests are declared beside each model rather than built as separate packages.
08 Reconciliation	3	1.0	Reconciliation compares outputs, which costs the same on either platform.
09 Documentation	1	0.7	dbt generates documentation and lineage from the code; only descriptions are written.
10 Deployment	2	0.9	The dbt project deploys as one unit; cut-over and hypercare remain.

TABLE 4.5 Base hours at level 3, before calibration, and the target platform's relative effort for each task. Each factor is an assumption, stated with its reason.

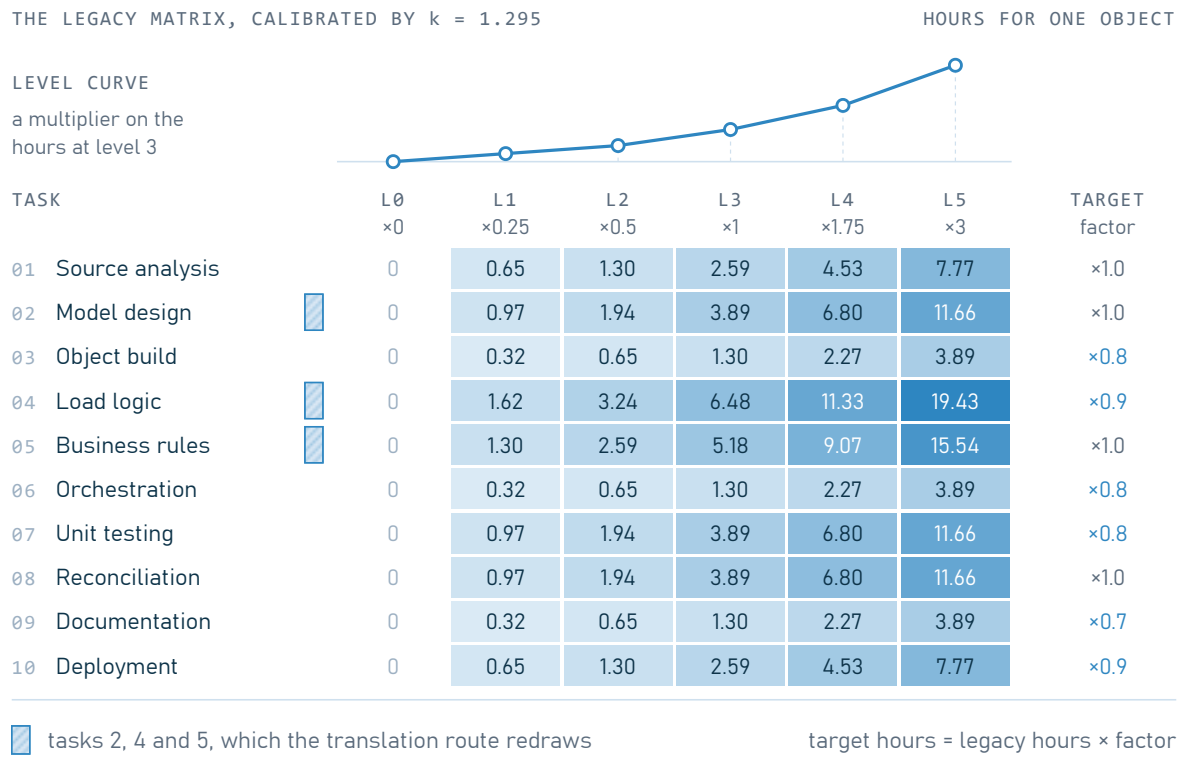


FIGURE 4.3 The calibrated legacy matrix: hours for one object at each task and level, with the level curve drawn over its columns and the target factor for each task beside its row.

The three matrices are defined by three short formulas. ROUND is Excel's, which rounds half away from zero; the model reproduces it so that the paper, the model and the workbook agree to the cent.

```
H0[t][c]  = B[t] * lam[c]                      uncalibrated, for calibration only
H_L[t][c] = ROUND(B[t] * lam[c] * k, 2)        legacy profile, lift and shift
H_X[t][c] = ROUND(B[t] * lam[c] * k * f[t], 2) target profile, translation

lam = [0.00, 0.25, 0.50, 1.00, 1.75, 3.00]       k = 1.295
```

4.6 Aggregation and the architectural component

Effort for a route is the sum, over every object and every task, of the matrix hours for that route's score. Converting at 8 productive hours a man-day gives the route in man-days. The translation premium is the ratio of the two routes less one, and the architectural component isolates the part of the difference that sits in the three translation tasks.

```
h(o, route) = sum over t of H_route[t][ s(o, t, route) ]       one object
E(route)    = sum over o of h(o, route)                       one route, hours
MD(route)   = E(route) / 8                                    man-days
premium     = MD(X) / MD(L) - 1
A_arch      = sum over o, over t in 2, 4, 5 of ( H_X[t][s(o,t,X)] - H_L[t][s(o,t,L)] ) / 8
```

Reporting the architectural component separately answers the question a steering group asks first: whether the premium is the price of the new pattern or the price of doing everything again. Chapter 6 shows that it is the former.

4.7 One object through both routes

The example object

OBJ-088: a satellite in the product area with 12 attributes, 1 source, 2 business rules and 3 dependants, keeping full history.

An aggregate hides the mechanism, so the paper follows one object through both routes. OBJ-088 was chosen by a fixed rule: among the satellites that keep full history and have between 12 and 30 attributes, it is the one with the median translation hours. Figure 4.4 explodes it through the ten tasks. Each plate is a task, and its thickness is the hours that task costs on that route.

The object costs 15.88 hours to lift, or 1.99 man-days, and 23.11 hours to translate, or 2.89 man-days: a premium of 45.5 per cent. The difference of 7.23 hours is not spread evenly. The three translation tasks add 8.10 hours, almost all of it in model design, which is not needed at all on lift and shift, and in load logic, which rises from level 1 to level 3. The other seven tasks give back 0.87 hours, because the target platform builds, schedules, tests, documents and deploys the object more cheaply. Table 4.6 shows the arithmetic behind every plate.

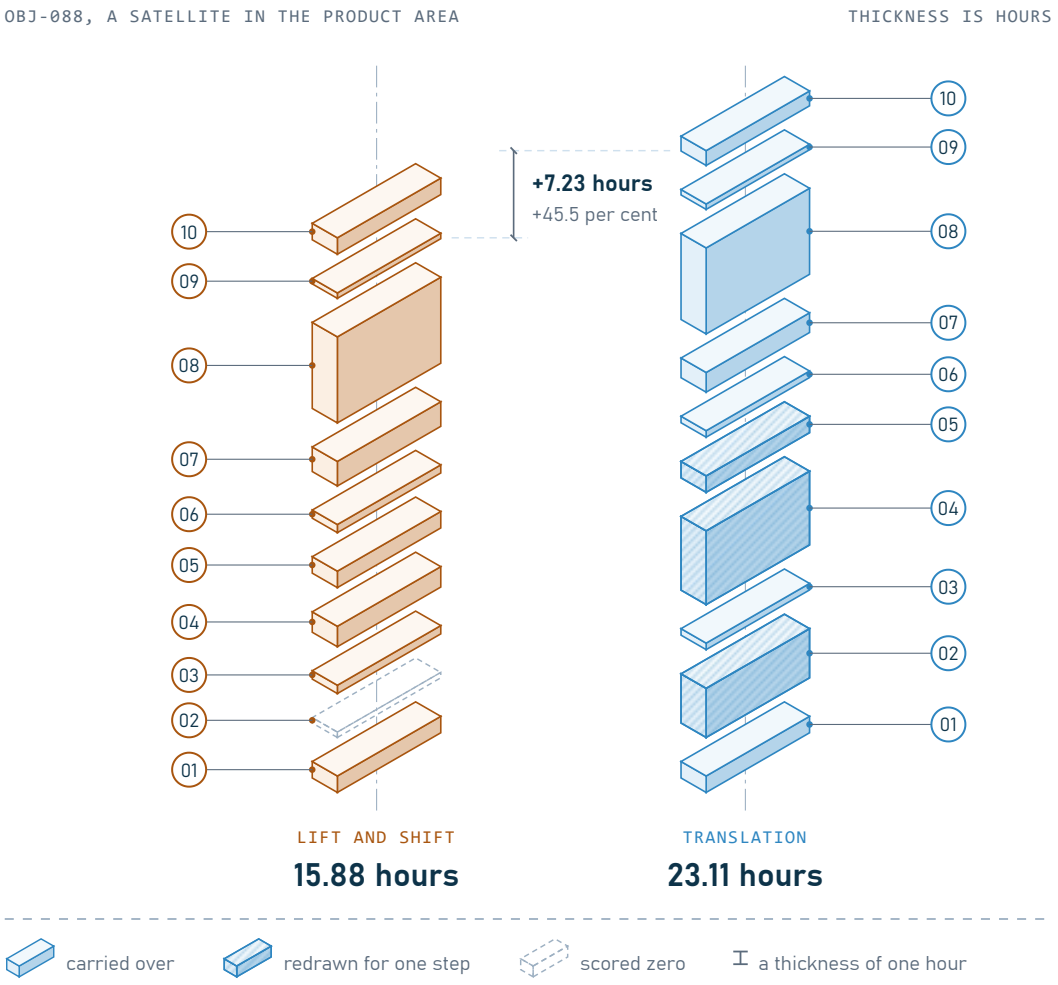


FIGURE 4.4 OBJ-088 exploded through the ten tasks on both routes. A hatched plate is a task redrawn for the single-step pattern; a dashed outline is a task scored zero.

Task	Lift	Lift hours	Trans.	Translation hours
01 Source analysis	2	2 × 0.50 × 1.295 = 1.30	2	2 × 0.50 × 1.295 × 1.0 = 1.30
02 Model design	0	0	3	3 × 1.00 × 1.295 × 1.0 = 3.89
03 Object build	2	1 × 0.50 × 1.295 = 0.65	2	1 × 0.50 × 1.295 × 0.8 = 0.52
04 Load logic	1	5 × 0.25 × 1.295 = 1.62	3	5 × 1.00 × 1.295 × 0.9 = 5.83
05 Business rules	1	4 × 0.25 × 1.295 = 1.30	1	4 × 0.25 × 1.295 × 1.0 = 1.30
06 Orchestration	2	1 × 0.50 × 1.295 = 0.65	2	1 × 0.50 × 1.295 × 0.8 = 0.52
07 Unit testing	2	3 × 0.50 × 1.295 = 1.94	2	3 × 0.50 × 1.295 × 0.8 = 1.55
08 Reconciliation	4	3 × 1.75 × 1.295 = 6.80	4	3 × 1.75 × 1.295 × 1.0 = 6.80
09 Documentation	1	1 × 0.25 × 1.295 = 0.32	1	1 × 0.25 × 1.295 × 0.7 = 0.23
10 Deployment	2	2 × 0.50 × 1.295 = 1.30	2	2 × 0.50 × 1.295 × 0.9 = 1.17
Total		**15.88**		**23.11**

TABLE 4.6 The arithmetic behind each plate: base hours, times the level multiplier, times k, times the target factor on translation, rounded to two decimals as the matrix cell is.

CHAPTER FIVE

05 Calibration

The matrix was written from judgement. This chapter measures it against what the original build of ATLAS actually cost, and corrects it.

5.1 The reference class

The base hours in Table 4.5 are a reconstruction of judgement: the hours an experienced engineer would expect a task to take at a moderate level of difficulty. Chapter 2 explained why such judgement is biased towards optimism, and why the remedy is to compare it with outcomes. The closest outcomes available are those of the original build of ATLAS on premises, for which the team kept timesheets per object.

Each sampled object was scored for its original build, a third route alongside lift and shift and translation in which every object is built from nothing on the legacy stack. The uncalibrated matrix converts those build scores into predicted hours, which can be set against the hours actually booked.

5.2 The stratified sample

The sample holds 45 objects, stratified by type so that each type is represented in proportion to its share of the inventory, with the places left over by rounding given to the types with the largest remainders. Table 5.1 shows the allocation and, in its last column, the diagnostic discussed in section 5.6.

Object type	Inventory	Sample	Actual over predicted
Staging table	68	11	1.195
Hub	33	5	1.059
Link	30	5	1.320
Satellite	68	10	1.321
Reference table	24	4	1.353
Mart or view	63	10	1.345
All types	**286**	**45**	**1.295**

TABLE 5.1 The calibration sample by type, and the ratio of actual to predicted hours within each type.

5.3 The ratio estimator

The calibration factor is the ratio of the total actual hours to the total predicted hours across the sample. This ratio estimator is the least-squares fit of a line through the origin when the scatter of actual hours grows in proportion to the predicted hours, which is the behaviour effort data usually show: a small object misses by a little and a large one by a lot (Cochran, 1977).

```
P(o) = sum over t of H0[t][ s(o, t, build) ]      predicted hours, uncalibrated
k    = ROUND( sum of A(o) / sum of P(o), 3 )       A(o) = actual hours booked
MAPE = 100 / n * sum of | A(o) - P(o) | / A(o)     before; k * P(o) after
bias = 100 * ( sum of P - sum of A ) / sum of A    negative means too low
```

5.4 Bias and error

-22.8%	1.295	20.3%	11.8%
BIAS BEFORE CALIBRATION	CALIBRATION FACTOR K	ERROR BEFORE	ERROR AFTER

Scored as they had been built, the 45 objects were predicted to take 831.8 hours. They took 1,076.9. The uncalibrated matrix was 22.8 per cent short, which is the planning fallacy expressed as a number: the expert-written matrix underestimated the original build by almost a quarter. Dividing the actual total by the predicted total gives k = 1.295, and multiplying every prediction by k brings the mean absolute percentage error down from 20.3 to 11.8 per cent.

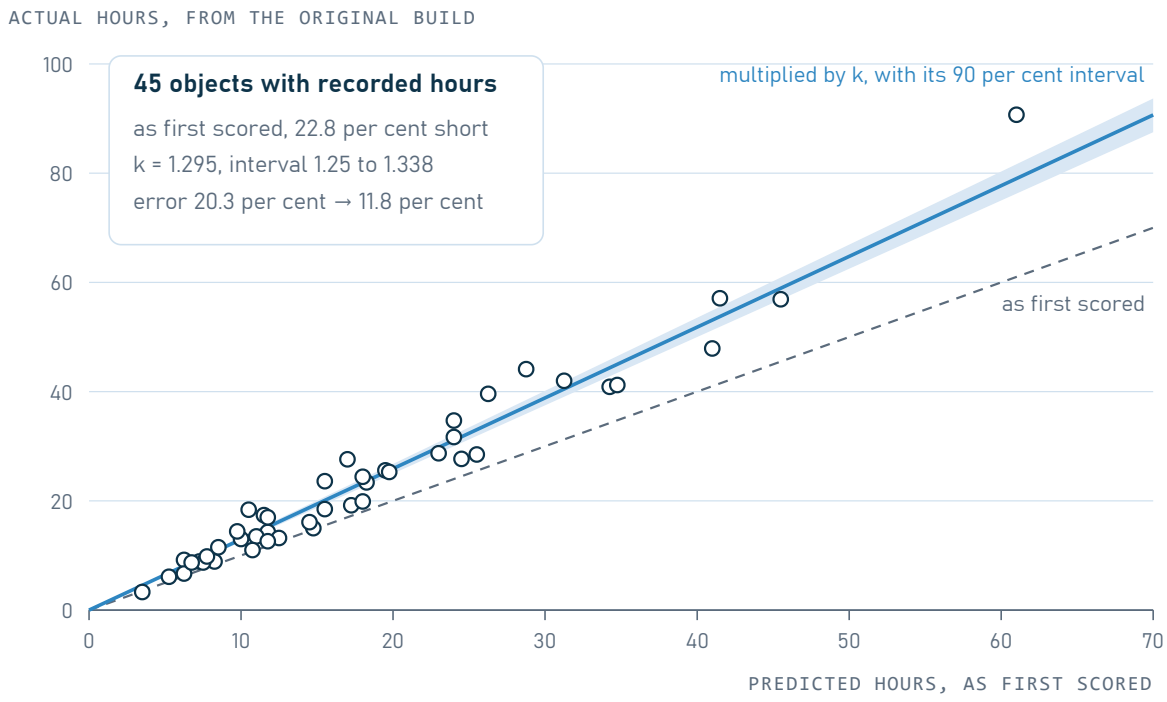


FIGURE 5.1 Predicted against actual hours for the calibration sample. The dashed line is the matrix as first written; the solid line is the same matrix multiplied by k, and the shaded wedge spans its 90 per cent bootstrap interval.

5.5 The bootstrap interval

A single factor estimated from 45 objects carries sampling error, and the estimate should say how much. Resampling the 45 objects with replacement 2,000 times, and recomputing the ratio each time, gives a distribution of k whose 5th and 95th percentiles are 1.25 and 1.338 (Efron and Tibshirani, 1993). The factor is therefore known to within 3.5 per cent either way, and Chapter 8 shows that an error of that size barely moves the answer.

5.6 The diagnostic by type

The last column of Table 5.1 repeats the ratio within each type. It is a diagnostic, not a second calibration, and it shows a pattern: links, satellites, reference tables and marts or views ran above the single factor, while staging tables and hubs ran below it. Marts and satellites, the types with the most logic and the most history, are the ones the matrix underestimates most.

The paper keeps one factor deliberately. Fitting six factors to 45 objects would fit noise as readily as signal, and some types have only four or five objects in the sample. The pattern is reported as a limitation and as the first refinement to make once the tracking described in Chapter 9 has produced a larger sample: calibration per type, or per type and route.

5.7 The calibrated and target matrices

With k fixed, the legacy matrix follows directly, and the target matrix follows from it through the factors in Table 4.5. Two cells show the arithmetic. For load logic at level 3 on the legacy profile, ROUND(5 × 1.00 × 1.295, 2) = 6.48 hours. For the same cell on the target profile, ROUND(5 × 1.00 × 1.295 × 0.9, 2) = 5.83 hours. Appendix A.3 prints all three matrices in full.

CHAPTER SIX

06 Results

Two totals, the difference between them taken apart by task, type and subject area, and the calendar time each route needs.

6.1 Two totals

521.6	675.6	154.0	29.5%
MAN-DAYS, LIFT AND SHIFT	MAN-DAYS, TRANSLATION	DIFFERENCE	TRANSLATION PREMIUM

Man-day
A man-day is 8 productive hours. The conversion is a parameter of the model, not a finding.

Summed over all 286 objects, lift and shift comes to 4,172.8 hours, or 521.6 man-days, and translation to 5,404.4 hours, or 675.6 man-days. Translation costs 154.0 man-days more, a premium of 29.5 per cent over rehosting the warehouse as it stands.

6.2 Where the difference sits: by task

Figure 6.1 takes the two totals apart by task. Most tasks cost much the same on either route; the three translation tasks do not.

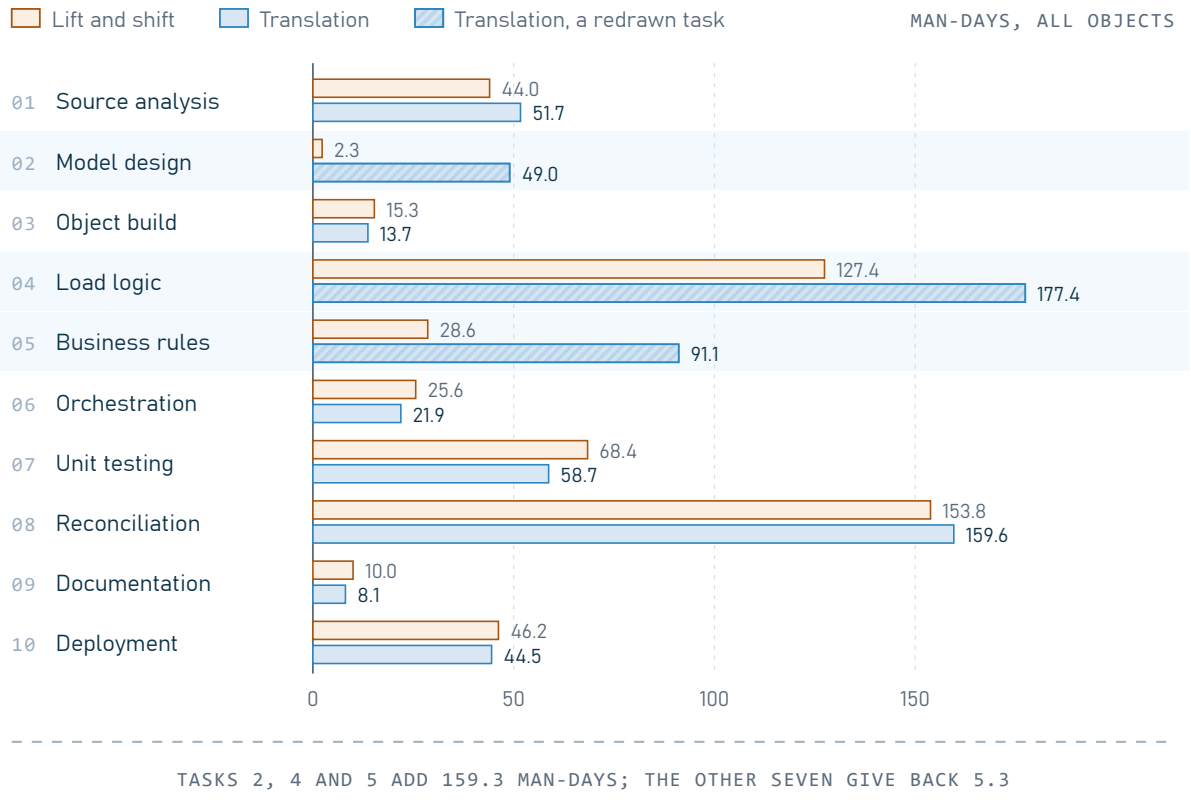


FIGURE 6.1 Effort by task for each route, side by side, in man-days across the whole inventory. The hatched bars are the three tasks the translation route redraws.

Model design rises from 2.3 to 49.0 man-days, load logic from 127.4 to 177.4 and business rules from 28.6 to 91.1. Together the three tasks add 159.3 man-days, which is 103.5 per cent of the whole difference, while the other seven tasks give back 5.3. The architectural component exceeds the difference because the target platform is marginally more efficient for the remaining work. The whole premium therefore sits in model design, load logic and business rules, which is the fact a choice between the routes most needs.

6.3 By object type

Object type	Objects	Lift and shift	Translation	Premium
Satellite	68	141.7	232.6	+64.1%
Link	30	48.7	63.7	+30.9%
Mart or view	63	142.7	183.6	+28.7%
Hub	33	47.4	59.5	+25.4%
Reference table	24	39.1	38.8	-0.7%
Staging table	68	102.0	97.3	-4.6%
All objects	**286**	**521.6**	**675.6**	**+29.5%**

TABLE 6.1 Man-days by object type on both routes, ordered by premium.

Satellites carry the largest premium, 64.1 per cent, because their history must be redesigned and their load logic rewritten. Staging tables and reference tables come out slightly cheaper on the target platform, because there is little to redesign in them and the platform's tooling saves time on the tasks they do need.

6.4 By subject area

Because the areas mix the types differently, their premiums differ too. They run from 12.3 per cent for reference data to 44.0 per cent for product, as Figure 6.2 shows. The areas rich in satellites sit at the top; the areas made mostly of staging tables, marts and reference tables sit at the bottom.

6.5 A Pareto view of the objects

Effort is not spread evenly across the objects either. Ranking the objects by their translation hours, the largest 58, which are 20 per cent of the inventory, carry 39.0 per cent of the translation effort (Figure 6.3). The curve is less steep than the familiar eighty-twenty rule, which matters for planning: no small set of objects can be singled out to make the estimate right, and the long tail of modest objects still carries most of the work.

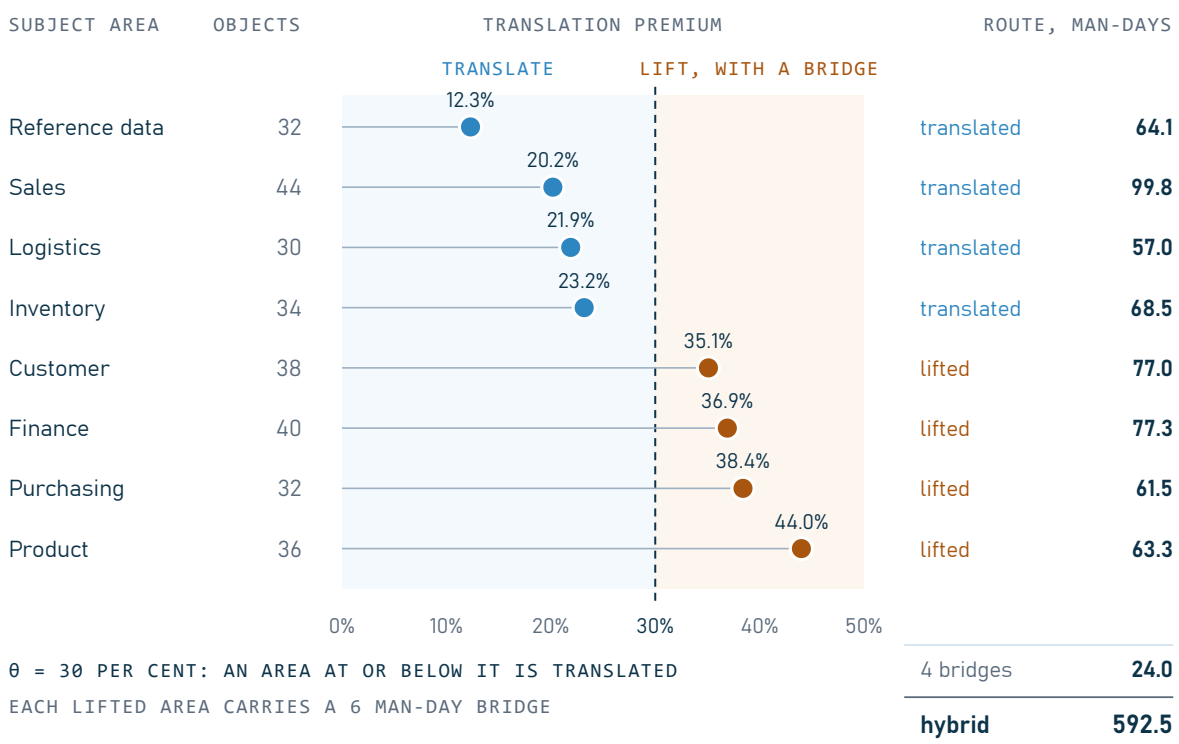


FIGURE 6.2 The translation premium by subject area, with the threshold that Chapter 7 uses to choose a route for each area and the man-days each area costs on its chosen route.

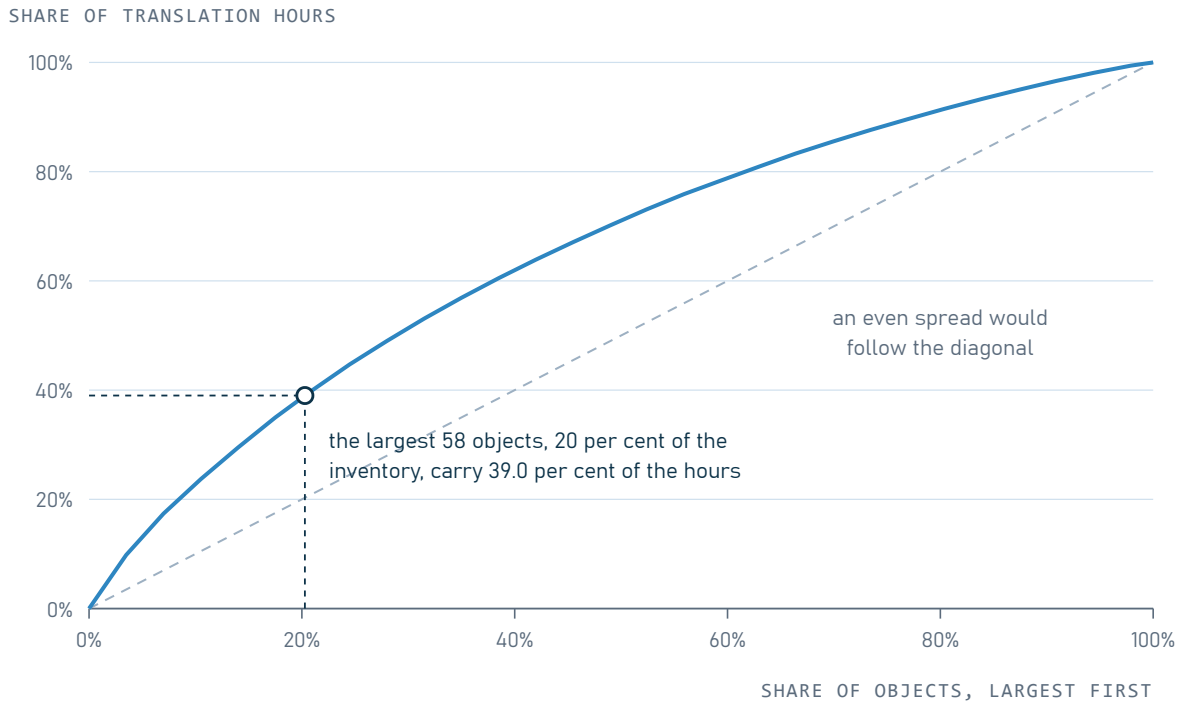


FIGURE 6.3 The Pareto curve of objects: the cumulative share of translation hours against the share of objects, largest first.

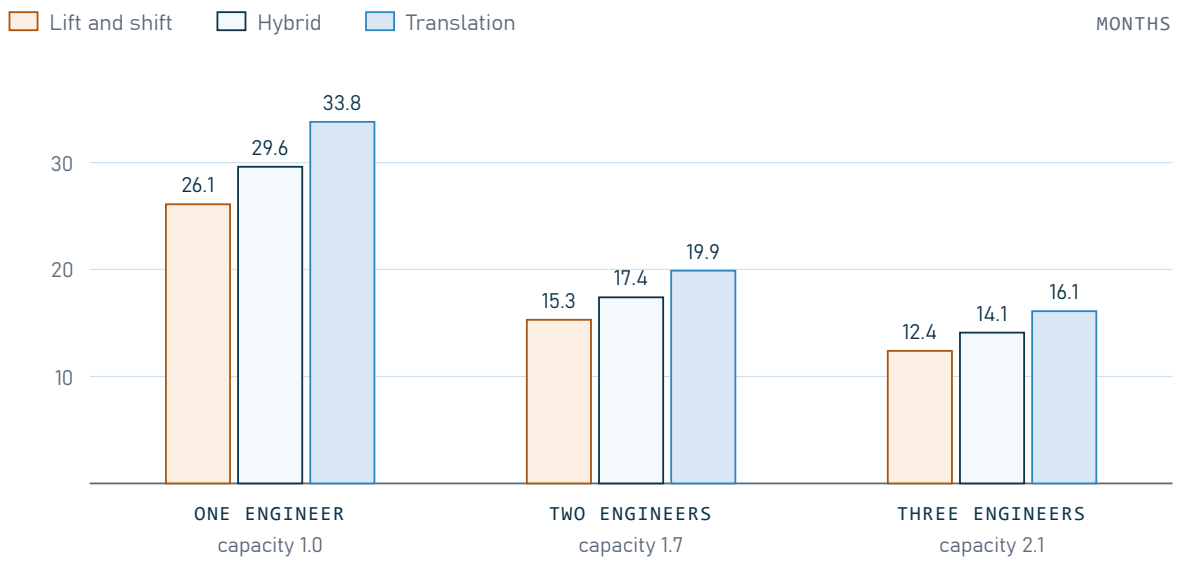
6.6 Duration and staffing

Calendar time follows from the man-days at 20 working days a month. One engineer is the simplest case. More engineers shorten the work, but not in proportion, because each person added must coordinate with the others. Brooks (1995) made the point for software projects in general; the model states it as a parameter, a loss of 15 per cent of capacity for each engineer added, so that two engineers deliver 1.7 engineers' worth of work and three deliver 2.1.

```
capacity(n)   = n * (1 - 0.15 * (n - 1))
months(MD, n) = MD / (20 * capacity(n))
```

Route	Man-days	One	Two	Three	Two to three
Lift and shift	521.6	26.1	15.3	12.4	12.4 to 15.3
Translation	675.6	33.8	19.9	16.1	16.1 to 19.9
Hybrid	592.5	29.6	17.4	14.1	14.1 to 17.4

TABLE 6.2 Months for one, two and three engineers. The hybrid route is derived in Chapter 7.



Each engineer added loses 15 per cent of capacity to coordination, so two engineers deliver 1.7 and three deliver 2.1.

FIGURE 6.4 Duration against team size for the three routes.

With one engineer, lift and shift takes 26.1 months and translation 33.8. With two or three engineers the ranges are 12.4 to 15.3 months and 16.1 to 19.9 months. The paper reports the single figure for one engineer and the range for two to three, because the team the programme will actually have is itself uncertain.

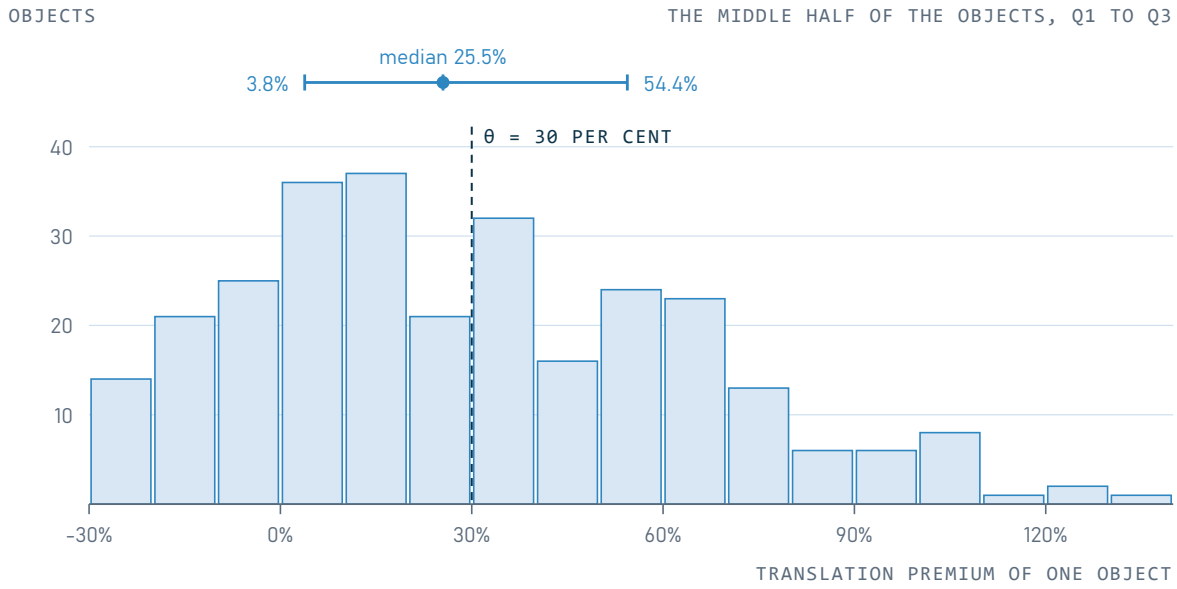
CHAPTER SEVEN

07 The hybrid route

An uneven premium invites a third route: translate where translation is cheap, lift where it is not, and say plainly what the choice defers.

7.1 The premium object by object

The premium of the estate as a whole hides a wide spread. Object by object, the median premium is 25.5 per cent, but half of all objects lie between 3.8 and 54.4 per cent, and 60 objects cost less to translate than to lift. 46.2 per cent of the objects carry a premium above 30 per cent (Figure 7.1).



46.2 per cent of the objects lie above the threshold, and 60 cost less to translate than to lift.

FIGURE 7.1 The per-object premium distribution, as supporting evidence for the decision rule: the translation premium of each object, with the middle half of the objects marked above.

7.2 The unit of decision

The decision cannot be taken object by object, because an object cannot live on one platform while the objects it depends on live on the other without a bridge between them for every dependency. The smallest unit that can move to one platform without a bridge inside it is the subject area, so the rule is applied to subject areas, and a bridge is costed for every boundary the rule creates.

7.3

The decision rule

Bridge
6 man-days per lifted area: the interface that exposes its marts to the translated estate, with its reconciliation.

A subject area is translated when its premium is at most 30 per cent, and lifted otherwise. Every lifted area pays a bridge of 6 man-days, and the refactoring it postpones, the difference between its translation and lift effort, is reported separately as a later decision rather than hidden in the total. Table 7.1 applies the rule.

Subject area	Objects	Lift	Translate	Premium	Route	Hybrid
Reference data	32	57.1	64.1	12.3%	Translate	64.1
Sales	44	83.0	99.8	20.2%	Translate	99.8
Logistics	30	46.8	57.0	21.9%	Translate	57.0
Inventory	34	55.6	68.5	23.2%	Translate	68.5
Customer	38	77.0	104.1	35.1%	Lift	83.0
Finance	40	77.3	105.8	36.9%	Lift	83.3
Purchasing	32	61.5	85.2	38.4%	Lift	67.5
Product	36	63.3	91.1	44.0%	Lift	69.3
All areas	**286**	**521.6**	**675.6**	**29.5%**		**592.5**

TABLE 7.1 The decision rule applied to each subject area, in man-days. The hybrid column includes the bridge for each lifted area.

7.4

The hybrid estimate

The rule translates sales, inventory, logistics and reference data and lifts customer, product, purchasing and finance. The hybrid comes to 592.5 man-days: 289.4 for the translated areas, 279.1 for the lifted areas and 24.0 for 4 bridges. It puts 49.0 per cent of the objects on the target pattern at once (Figure 7.2).

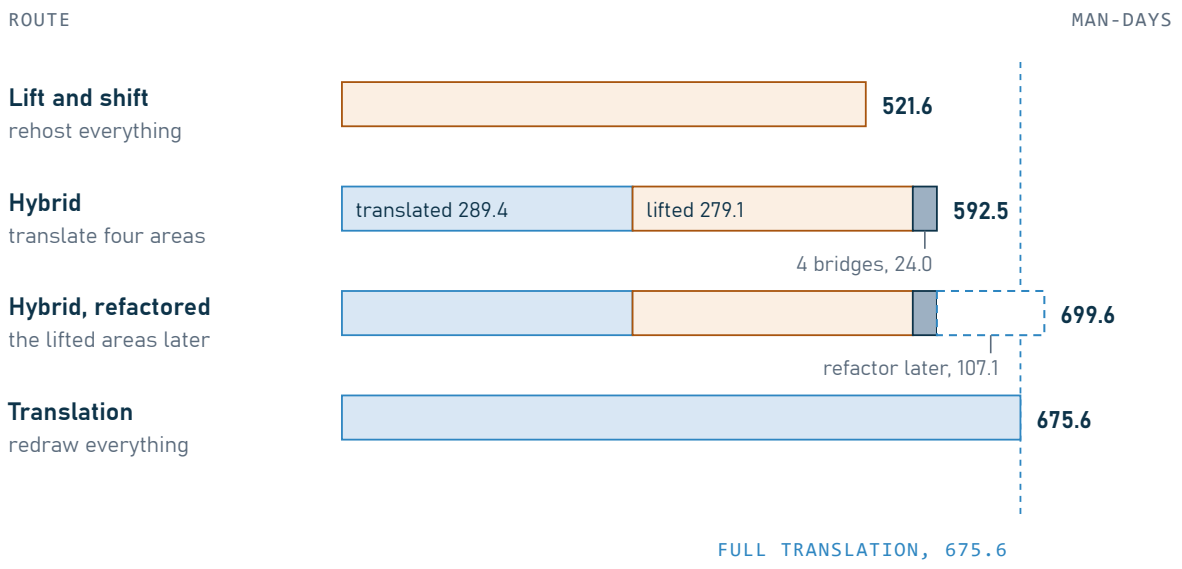


FIGURE 7.2 The hybrid allocation: the hybrid's man-days by part, and the effect of refactoring the lifted areas later, against the two original routes.

The honest conclusion follows from these figures. The hybrid costs 70.9 man-days more than lift and shift and 83.1 fewer than full translation, and it converges about half of the estate at once. If the lifted areas are refactored later, at 107.1 man-days, the total comes to 699.6, which exceeds full translation by 24.0, the cost of the bridges. The hybrid therefore buys earlier delivery and lower immediate risk, not a lower total.

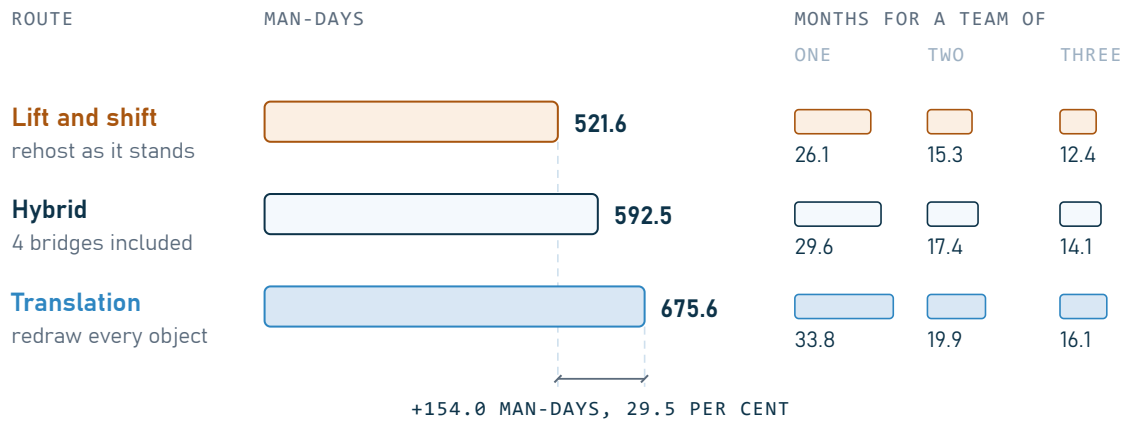


FIGURE 7.3 The three routes in man-days and in months for a team of one, two or three engineers.

7.5 What each route buys and costs

Table 7.2 sets the three routes against one another on the measures the steering group weighed. The case recommends the hybrid, assessed on these measures and built on the complexity estimate. It is not a compromise between two camps; it is the route the estimate itself points to once the premium is seen to be uneven.

Route	Man-days	Months, two	Converged	What it buys	What it defers
Lift and shift	521.6	15.3	None	The fastest exit from the legacy hosting	All of the convergence
Hybrid	592.5	17.4	49.0%	Convergence where it is cheap, at once	107.1 man-days of refactoring
Translation	675.6	19.9	All	Full convergence and one pattern	Nothing, at the highest cost now

TABLE 7.2 What each route buys and what it defers.

CHAPTER EIGHT

08 Sensitivity and limitations

Which inputs move the answer, which do not, and what the method cannot see however carefully it is applied.

8.1 One metric

Sensitivity is tested on one metric for every factor, so that the factors can be compared: the duration of the hybrid route with two engineers, in months, with the allocation of subject areas held fixed. Its base value is 17.4 months.

8.2 Five cases

Factor	How it is applied	Low	High	Swing
Every non-zero score one level lower or higher	Every non-zero score on both routes moves one level, held between 1 and 5	11.0	30.0	19.0
Matrix hours 20 per cent lower or higher	Every matrix cell multiplied by 0.8 or 1.2; bridges unchanged	14.1	20.8	6.7
Productive hours per man-day 8 (base) or 6.5	Man-days computed at 6.5 productive hours instead of 8	17.4	21.3	3.9
Coordination overhead 10 or 20 per cent per added engineer	Capacity loss of 0.10 or 0.20 for the second engineer	16.5	18.5	2.0
Calibration factor at its 90 per cent bootstrap bounds	Every cell multiplied by the lower or upper bound over k	16.8	18.0	1.2

TABLE 8.1 The five sensitivity cases, in months for the hybrid route with two engineers.

8.3 Why the scores dominate

Moving every non-zero score one level lower or higher swings the duration from 11.0 to 30.0 months, a swing of 19.0. The calibration factor at either end of its interval moves it only from 16.8 to 18.0. The score shift is also asymmetric: one level higher adds 12.6 months while one level lower removes 6.4, because the level curve is convex and each step up costs more than the step below it.

The practical consequence is that review time belongs on the scores. A systematic bias in the scorer, one level too optimistic across the board, is the largest single source of error in the method, larger than any uncertainty in the matrix or the calibration. Section 8.5 describes the defence against it.

8.4 What the method cannot see

Three kinds of error lie outside the method's reach, and each needs to be named so that it is not mistaken for something the estimate has covered.

- **Unknown unknowns.** An object that is not in the catalogue, a source nobody remembered, or a dependency that appears only at cut-over is not in the inventory and so not in the estimate. The inventory query and the dependency analysis reduce this risk; they cannot remove it.
- **Programme-level work.** Work that belongs to no object is outside the object model by design, and is estimated separately, from the top down, as Table 8.2 shows.
- **Scorer bias.** The scores are judgements anchored to drivers, and the same anchoring that makes them consistent can make them consistently wrong.

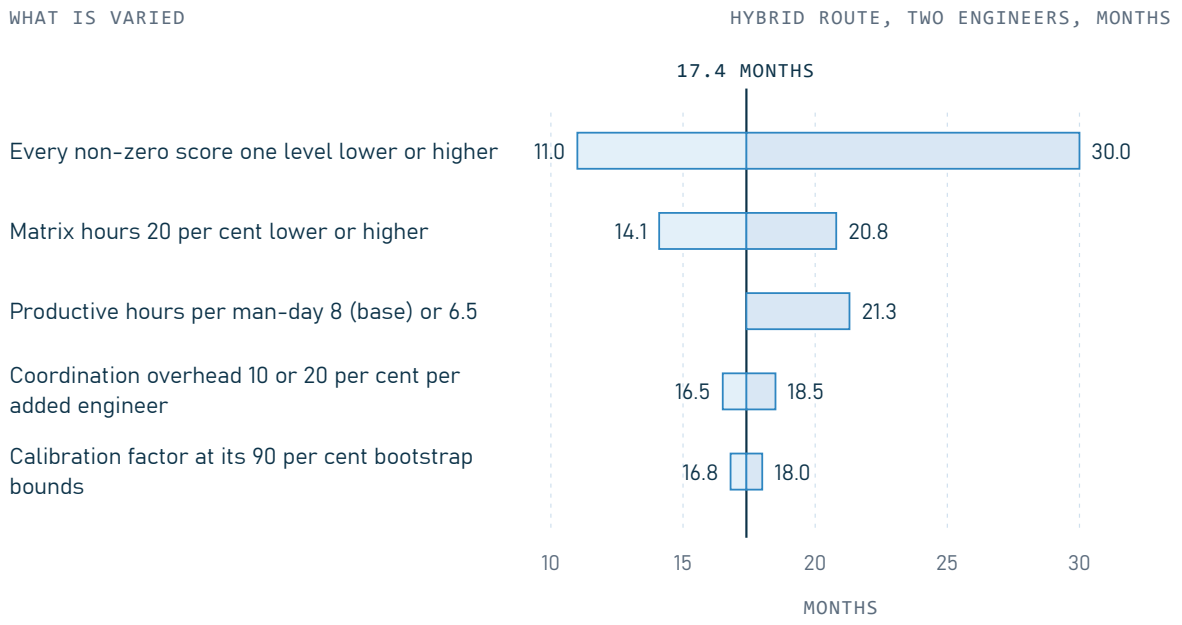


FIGURE 8.1 The sensitivity tornado: the hybrid route's duration for two engineers as each input moves between its low and high case.

Programme-level work is kept apart for a reason the earlier paper on top-down and bottom-up estimation set out. Spreading it across objects as a percentage would hide it inside numbers that nobody could then challenge, and would make it grow and shrink with the object count, which it does not. The case does not price these items, because the comparison between routes does not depend on most of them; Table 8.2 lists them with the basis on which each should be estimated.

8.5 Two scorers

The defence against scorer bias is the one forecasting research recommends for any judgement that matters: a second, independent judgement, and a review of where the two disagree. Each object is scored by two people who do not see each other's scores. Where their scores for a task differ by more than one level, the two meet, read the drivers and the rubric together, and record the score they agree with the reason for it. Where they differ by one level, the higher score is taken, because the level curve punishes optimism more than caution.

Item	Why it is outside the object model	How it is estimated	Differs by route
Environments	It belongs to the platform, not to any object	From past set-ups of the same platform	Yes: route A hosts SQL Server and SSIS
Security and access	It applies to the estate as a whole	Per role model and data domain	Little
Deployment pipelines	They are built once for the estate	Per pipeline	Yes: SSIS and dbt deploy differently
Programme management	It scales with duration and team, not objects	A share of the duration, by convention	With the route's duration
Training and handover	It scales with people, not objects	Per team	Yes: route B teaches a new pattern

TABLE 8.2 Programme-level items, estimated top-down and kept outside the object model.

The cost is modest. Scoring an object takes minutes once its drivers are known, and a second scorer roughly doubles minutes rather than days. Set against the swing of 19.0 months that a one-level bias would cause, the second scorer is the cheapest insurance the estimate can buy.

09

CHAPTER NINE

Governing the estimate

An estimate is not finished when it is approved. It is kept, revised and measured against the work, and it earns trust by surviving that.

9.1 The score record

Every score is a record, not a cell in a spreadsheet that someone once filled in. The record says which object, task and route the score belongs to, what the level is, who set it, when, and why, and it is never overwritten: a revised score is a new record that points to the one it replaces. Table 9.1 lists the fields.

Field	Example	Why it is kept
Object, task, route	OBJ-088, task 4, translation	The address of the score in the estimate
Level	3	The score itself
Scorer and date	Two scorers, the date each scored	Who to ask, and when it was true
Reason	Full-history satellite; two-step load redrawn as one model	What a reviewer checks against the drivers
Replaces	The record it supersedes, if any	The audit trail of revisions
Actual hours	Booked as the task is done	The evidence for recalibration

TABLE 9.1 The fields of a score record.

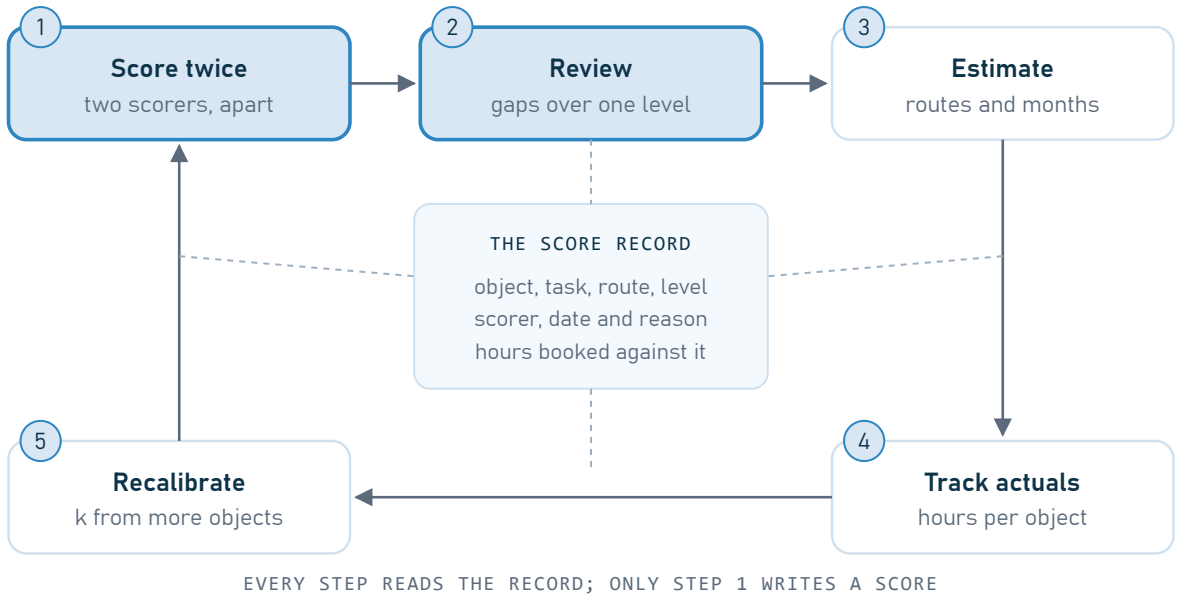


FIGURE 9.1 The governance loop, with the score record at its centre.

9.2 Re-scoring as discovery proceeds

Discovery changes what is known about objects, and the estimate should change with it. When analysis finds a fourth source for a satellite, or a business rule nobody had documented, the affected scores are revised through the same two-scorer procedure, and the estimate is recomputed. Because every figure is a sum of scores, a revision moves the total by exactly the hours of the cells it changes, and the steering group can see which discovery moved the estimate and by how much.

9.3 Tracking actuals against the estimate

Once work begins, hours are booked against the object and task they were spent on, which is the same address the score has. That habit does two things. It shows early, object by object, where the estimate is wrong, long before the total drifts far enough to be noticed. And it builds the larger sample that section 5.6 asked for, from which k can be recomputed and, once the sample allows it, calibrated per type.

9.4 The estimate in a steering decision

The choice between lift and shift and translation was a contested architectural choice, and contested choices tend to be argued as positions. An estimate built from scored objects changes the form of the argument. A person who believes translation is underestimated must say where: which objects, which tasks, which scores. That claim can be checked against the drivers and the rubric, and either the score changes or the claim falls. The decision then rests on an estimate that both sides have been able to inspect line by line, which is the most that any estimate can honestly offer.

CHAPTER TEN

10

Conclusion

What the estimate found, and what the method is for.

Northmere West faced a choice between rehosting its warehouse and rebuilding it in the group's pattern. Estimated object by object across 286 objects and ten tasks, rehosting comes to 521.6 man-days and rebuilding to 675.6, a premium of 29.5 per cent. The whole premium sits in three tasks, model design, load logic and business rules, and it is uneven across the estate, from 12.3 to 44.0 per cent by subject area.

That unevenness produced a third route. The hybrid, at 592.5 man-days, converges 49.0 per cent of the objects at once, takes 14.1 to 17.4 months with two or three engineers, and defers the rest of the convergence as a decision of its own. It buys earlier delivery and lower immediate risk, not a lower total, and the paper says so because a recommendation that hides its cost does not survive its first review.

Calibration against the original build found the expert matrix 22.8 per cent short, and corrected it. The sensitivity analysis found that the answer depends far more on the scores than on the matrix or the calibration, which is why the method asks for two scorers and a record of every score.

None of the steps is clever. An inventory, a rubric, a matrix, a ratio and a sum are all ordinary. What the method offers is not precision but accountability: an estimate that can be taken apart, argued with in its own terms, revised as the facts arrive and measured against the work. A total that cannot be challenged will not be trusted for long, and an estimate built to be challenged keeps its credibility because it can be.

APPENDIX

Appendix

Definitions, matrices, an excerpt of the inventory, the formulas, a glossary and the references.

A.1 Task definitions

No	Task	Definition
01	**Source analysis and lineage**	Tracing each source of the object, its joins and filters, and the lineage from source to object, including the reading of existing code and the questions put to source owners.
02	**Target data model design**	Deciding the object's keys, granularity and historisation in the pattern of the route, and agreeing the design with the people who will maintain it. On lift and shift the existing design is kept, so the task is usually scored zero.
03	**Object build**	Creating the tables, views and their physical properties on the platform, by hand or by the platform's own materialisation.
04	**Load logic development**	Writing the mapping, package or model that loads the object, including incremental logic and the handling of history.
05	**Business rules and transformations**	Implementing and agreeing the business rules the object applies: derivations, classifications, filters and calculations.
06	**Orchestration and scheduling**	Placing the object's load in the job chain, with its dependencies, schedule and restart behaviour.
07	**Unit testing**	Testing the load and the logic of the object on its own, with prepared test cases.
08	**Data reconciliation against the source**	Comparing the object's output with its source, and with the legacy output on a migration, by counts, sums and history, and explaining every difference.
09	**Documentation and lineage registration**	Registering the object's business description, ownership and lineage in the catalogue.
10	**Deployment, cut-over and hypercare**	Deploying the object, cutting over from the legacy load, and supporting it through the first weeks of production.

TABLE A.1 The ten tasks in full.

A.2 The rubric and the build route

The rubric is given in Table 4.2 and the scoring guide for lift and shift and translation in Table 4.4. Table A.2 completes the guide with the build route, used only to score the calibration sample.

Task	Build route, calibration only
01 Source analysis and lineage	1.00 + 1.80 s + 0.90 d
02 Target data model design	1.00 + 0.90 a + 1.80 h
03 Object build	1.00 + 1.08 a
04 Load logic development	1.00 + 1.80 s + 1.08 h
05 Business rules and transformations	0.50 + 2.70 r
06 Orchestration and scheduling	1.00 + 1.80 d
07 Unit testing	1.00 + 0.72 a + 1.80 r
08 Data reconciliation against the source	1.00 + 0.90 h + 1.80 v
09 Documentation and lineage registration	1.00 + 0.72 a
10 Deployment, cut-over and hypercare	1.00 + 0.90 d

TABLE A.2 The scoring guide for the original build.

A.3 The lookup matrices

Task	Level 0	Level 1	Level 2	Level 3	Level 4	Level 5
01 Source analysis	0.00	0.50	1.00	2.00	3.50	6.00
02 Model design	0.00	0.75	1.50	3.00	5.25	9.00
03 Object build	0.00	0.25	0.50	1.00	1.75	3.00
04 Load logic	0.00	1.25	2.50	5.00	8.75	15.00
05 Business rules	0.00	1.00	2.00	4.00	7.00	12.00
06 Orchestration	0.00	0.25	0.50	1.00	1.75	3.00
07 Unit testing	0.00	0.75	1.50	3.00	5.25	9.00
08 Reconciliation	0.00	0.75	1.50	3.00	5.25	9.00
09 Documentation	0.00	0.25	0.50	1.00	1.75	3.00
10 Deployment	0.00	0.50	1.00	2.00	3.50	6.00

TABLE A.3 The uncalibrated legacy matrix, in hours, used only for calibration.

Task	Level 0	Level 1	Level 2	Level 3	Level 4	Level 5
01 Source analysis	0.00	0.65	1.30	2.59	4.53	7.77
02 Model design	0.00	0.97	1.94	3.89	6.80	11.66
03 Object build	0.00	0.32	0.65	1.30	2.27	3.89
04 Load logic	0.00	1.62	3.24	6.48	11.33	19.43
05 Business rules	0.00	1.30	2.59	5.18	9.07	15.54
06 Orchestration	0.00	0.32	0.65	1.30	2.27	3.89
07 Unit testing	0.00	0.97	1.94	3.89	6.80	11.66
08 Reconciliation	0.00	0.97	1.94	3.89	6.80	11.66
09 Documentation	0.00	0.32	0.65	1.30	2.27	3.89
10 Deployment	0.00	0.65	1.30	2.59	4.53	7.77

TABLE A.4 The calibrated legacy matrix, k = 1.295, used by lift and shift.

Task	Level 0	Level 1	Level 2	Level 3	Level 4	Level 5
01 Source analysis	0.00	0.65	1.30	2.59	4.53	7.77
02 Model design	0.00	0.97	1.94	3.89	6.80	11.66
03 Object build	0.00	0.26	0.52	1.04	1.81	3.11
04 Load logic	0.00	1.46	2.91	5.83	10.20	17.48
05 Business rules	0.00	1.30	2.59	5.18	9.07	15.54
06 Orchestration	0.00	0.26	0.52	1.04	1.81	3.11
07 Unit testing	0.00	0.78	1.55	3.11	5.44	9.32
08 Reconciliation	0.00	0.97	1.94	3.89	6.80	11.66
09 Documentation	0.00	0.23	0.45	0.91	1.59	2.72
10 Deployment	0.00	0.58	1.17	2.33	4.08	6.99

TABLE A.5 The target matrix, used by translation.

A.4 An inventory excerpt

Table A.6 shows the first twenty objects of the 286, with OBJ-088 added at the foot. The full inventory, with every score, is in the workbook.

TABLE A.6 The first twenty objects, and the worked example, with their drivers and their hours on each route.

Object	Area	Type	Attr	Src	Rules	History	Volume	Dep	DQ	Lift h	Trans h
OBJ-001	Sales	Mart or view	17	2	0	SCD2	Medium	2	Yes	21.70	25.50
OBJ-002	Sales	Mart or view	25	4	1	None	Medium	0	No	13.60	13.39
OBJ-003	Sales	Mart or view	12	4	4	SCD1	Low	0	No	15.22	16.81
OBJ-004	Sales	Mart or view	34	1	0	None	Medium	1	Yes	16.20	13.82
OBJ-005	Sales	Staging table	34	1	2	SCD1	Low	1	No	7.78	8.69
OBJ-006	Sales	Mart or view	107	7	17	SCD2	Medium	6	No	38.22	52.32
OBJ-007	Sales	Satellite	43	3	2	SCD1	Low	13	Yes	23.97	29.76

Object	Area	Type	Attr	Src	Rules	History	Volume	Dep	DQ	Lift h	Trans h
OBJ-008	Sales	Reference table	4	2	0	None	Medium	10	No	17.16	16.59
OBJ-009	Sales	Mart or view	92	10	1	None	Low	21	No	24.60	32.22
OBJ-010	Sales	Staging table	21	3	1	None	Medium	4	No	11.99	10.67
OBJ-011	Sales	Mart or view	75	4	2	SCD2	Low	8	No	16.86	25.81
OBJ-012	Sales	Staging table	25	1	1	None	Low	0	No	7.76	8.16
OBJ-013	Sales	Mart or view	36	5	6	None	Low	0	No	17.16	18.30
OBJ-014	Sales	Mart or view	24	1	3	None	Low	0	No	8.08	8.49
OBJ-015	Sales	Staging table	70	2	0	None	High	1	No	19.44	14.31
OBJ-016	Sales	Mart or view	58	10	5	SCD1	Low	13	No	21.38	34.47
OBJ-017	Sales	Satellite	10	1	1	SCD2	Medium	6	No	19.10	25.82
OBJ-018	Sales	Staging table	2	1	0	None	Medium	0	No	10.36	8.76
OBJ-019	Sales	Staging table	6	1	0	None	Low	0	No	2.91	3.50
OBJ-020	Sales	Satellite	16	1	1	SCD2	Medium	3	No	14.25	22.85
OBJ-088	Product	Satellite	12	1	2	SCD2	Medium	3	No	15.88	23.11

A.5 The formulas

The workbook computes every result with live formulas, so that a reader can change one score or one parameter and watch the totals move. The central formulas are these.

```
' matrix cell, task t and level c, on Lookup_Legacy
=ROUND(INDEX(Base,t)*INDEX(Lambda,c+1)*k,2)
' hours for task n of one object, on Scores_LiftShift
=INDEX(M_Legacy,n,Sn+1)
' calibration factor, on Calibration
=ROUND(SUM(A)/SUM(P),3)
' route for a subject area, on Hybrid
=IF(Premium<=Theta,"Translate","Lift and shift")
' months for a team of n
=MD/(DaysPerMonth*n*(1-Coord*(n-1)))
```

The same calculation in Python, from the reference model, and in SQL against a table of scores and a table of matrix cells, follows.

```
def hours(matrix, scores):
    return sum(matrix[t][scores[t]] for t in range(10))

md_lift  = sum(hours(H_L, o.s_lift)  for o in inventory) / 8
md_trans = sum(hours(H_X, o.s_trans) for o in inventory) / 8
```

```
SELECT s.route, SUM(m.hours) / 8.0 AS man_days
FROM   scores AS s
JOIN   matrix AS m
  ON   m.profile = CASE s.route WHEN 'lift' THEN 'legacy' ELSE 'target' END
 AND   m.task = s.task AND m.level = s.level
GROUP  BY s.route;
```

A.6 Glossary

TABLE A.7 Terms used in the paper.

Term	Meaning
Architectural component	The part of the translation premium carried by the three translation tasks.
Bridge	The interface between a lifted subject area and the translated estate.
Calibration factor, k	The ratio of actual to predicted hours on the calibration sample, applied to the whole matrix.
Data Vault 2.0	A warehouse modelling pattern of hubs, links and satellites, loaded in a raw step and a business step.
dbt	A tool that builds warehouse models from SQL statements, with tests and documentation.
Hub, link, satellite	Data Vault structures for business keys, relationships, and descriptive attributes with their history.
Hybrid route	Translating some subject areas and lifting the rest.
Level curve	The multipliers that scale base hours from level 0 to level 5.
Lift and shift	Rehosting a system on new infrastructure without changing its design.
MAPE	Mean absolute percentage error, the average size of the error relative to the actual.
Man-day	8 productive hours of one person's work.
Planning fallacy	The tendency to underestimate how long planned work will take.
Reference class	A set of comparable past cases whose outcomes are known.
Relative-effort factor	The target platform's effort for a task, relative to the legacy platform's at the same level.
SCD1, SCD2	Ways of keeping history: overwrite the old value, or keep every version.
Storage object	Anything the warehouse persists and names in its model.
Translation	Rebuilding a system in the pattern of its target platform.
Translation premium	The extra effort of translation over lift and shift, as a share of lift and shift.

A.7 References and further reading

Albrecht, A. J. (1979). Measuring application development productivity. In Proceedings of the Joint SHARE, GUIDE and IBM Application Development Symposium, Monterey, California, pp. 83–92.

Boehm, B. W., Abts, C., Brown, A. W., Chulani, S., Clark, B. K., Horowitz, E., Madachy, R., Reifer, D. J. and Steece, B. (2000). Software Cost Estimation with COCOMO II. Prentice Hall.

Brooks, F. P. (1995). The Mythical Man-Month: Essays on Software Engineering, anniversary edition. Addison-Wesley. First published 1975.

Buehler, R., Griffin, D. and Ross, M. (1994). Exploring the planning fallacy: why people underestimate their task completion times. Journal of Personality and Social Psychology, 67(3), 366–381.

Cochran, W. G. (1977). Sampling Techniques, third edition. Wiley.

Efron, B. and Tibshirani, R. J. (1993). An Introduction to the Bootstrap. Chapman and Hall.

Flyvbjerg, B. (2006). From Nobel Prize to project management: getting risks right. Project Management Journal, 37(3), 5–15.

Flyvbjerg, B. (2008). Curbing optimism bias and strategic misrepresentation in planning: reference class forecasting in practice. European Planning Studies, 16(1), 3–21.

Hussain, T. (2022). Top-down or bottom-up: which approach is better? Research paper.

International Function Point Users Group (2010). Function Point Counting Practices Manual, release 4.3.

Kahneman, D. and Tversky, A. (1982). Intuitive prediction: biases and corrective procedures. In D. Kahneman, P. Slovic and A. Tversky (eds), Judgment under Uncertainty: Heuristics and Biases, pp. 414–421. Cambridge University Press. First published 1979.

Linstedt, D. and Olschimke, M. (2015). Building a Scalable Data Warehouse with Data Vault 2.0. Morgan Kaufmann.

McConnell, S. (2006). Software Estimation: Demystifying the Black Art. Microsoft Press.

Project Management Institute (2017). A Guide to the Project Management Body of Knowledge (PMBOK Guide), sixth edition. Project Management Institute.

A.8 A note on the fictitious data

Northmere Components Group, Northmere West, ATLAS and HORIZON are invented, and so is every figure in this paper. The inventory, the scores and the calibration actuals were generated by the reference model from a fixed random seed, 20261101. Each object's drivers were drawn from distributions set per object type and scaled by one size factor per object, so that a large object is large on every driver. Its scores were produced by the scoring guide in Table 4.4 with a small random term for judgement. The calibration actuals were produced by multiplying each sampled object's predicted hours by a hidden factor per type and a random error; the model never sees the hidden factors, and section 5.6 shows how much of them the single calibration factor recovers. The methodology, by contrast, is real: it is the method as used in practice, applied here to an invented case so that it can be published and reproduced in full.